

CASSANDRA PYTHON DRIVER

ITESO
Universidad Jesuita
de Guadalajara



Profesora: Ing. Ivanna Haro

Integrantes:

Sofia Marcela Jiménez López

Haziel Isaías Murillo Echauri

Fecha: 13 de septiembre

Enlace al repositorio: <https://github.com/asiel-007/movies-sofia-haziel>

Creación de entorno virtual e instalación de paquetes

Una vez activado el ambiente virtual, se puede instalar el driver de Cassandra usando la utilería de pip. La instalación se puede realizar utilizando el archivo requirements.txt, que contiene la versión del driver que utilizará el script, siendo esta la más reciente.

```
(movies) hazmull@whale:~/iteso/fall26/nrdb/movies (main)$ uv pip freeze
cassandra-driver==3.30.1
click==8.5.0
deprecated==1.3.1
geomet==1.1.0
wrapt==2.4.0
```

De esta forma, la estructura del proyecto termina siendo la siguiente:

```
(movies) hazmull@whale:~/iteso/fall26/nrdb/movies (main)$ tree -a -L 1
.
├── .git
├── .gitignore
├── .venv
├── movies.py
└── requirements.txt
```

Configurar Cassandra y el Keyspace

La creación del keyspace es primordial ya que funciona como el contenedor principal porque define cómo se replican, distribuyen y organizan los datos en todo el clúster

```
CREATE_KEYSPACE = """
CREATE KEYSPACE IF NOT EXISTS movies
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'};
"""
```

Crenando las dos tablas principales de películas tanto por título como por género

```
def create_keyspace_and_tables(session):
    session.execute(CREATE_KEYSPACE)
    session.set_keyspace("movies")
    title_stmt = session.prepare(CREATE_TABLE_MOVIE_BY_TITLE)
    session.execute(title_stmt)
    genre_stmt = session.prepare(CREATE_TABLE_MOVIE_BY_GENRE)
    session.execute(genre_stmt)
```

Cassandra ejecutándose mediante Docker

Para crear el contenedor de docker se utiliza la imagen más reciente de Cassandra, y se mapea el puerto 9042 de TCP del contenedor con el mismo puerto del host para que la aplicación pueda acceder a la instancia de Cassandra del contenedor.

```
(movies) hazmull@whale:~/iteso/fall26/nrdb/movies (main)$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED
192acea7ebda   cassandra:latest  "docker-entrypoint.s..." 2 weeks ago

PORTS          NAMES
7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp  cassandra-db s
```

Creación de tablas para diferentes patrones de acceso

Para esta actividad crearás dos tablas, cada una optimizada para un patrón de acceso diferente.

Ambas tablas tendrán las siguientes columnas:

Columna	Tipo	Descripción
movie_id	UUID	Identificador único de la película
title	TEXT	Título de la película
release_year	INT	Año de estreno
genre	TEXT	Género de la película
rating	FLOAT	Calificación promedio
director	TEXT	Director de la película

Tabla 1: movie_by_title

Este diseño permite almacenar películas que tengan el mismo título pero diferentes años de estreno dentro de la misma partición, son ordenadas de manera descendiente por el año de estreno. Siendo su primary key title y release_year.

```
CREATE_TABLE_MOVIE_BY_TITLE = ""
CREATE TABLE IF NOT EXISTS movies_by_title (
    movie_id UUID, title TEXT,
    release_year INT, genre TEXT,
    rating FLOAT, director TEXT,
    PRIMARY KEY (title, release_year)
) WITH CLUSTERING ORDER BY (release_year DESC);
""
```

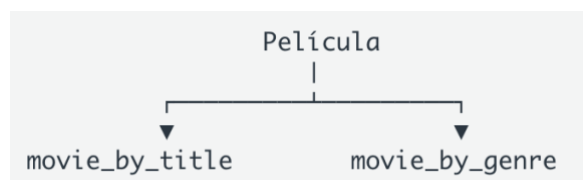
Tabla 2: movie_by_genre

Esta tabla se utilizará para consultar películas por género y obtenerlas ordenadas por su calificación. La clustering key permitirá ordenar las películas dentro de cada género de acuerdo con su calificación. Al igual es importante mencionar la ampliación de la primary key con el uso release_year para lograr identificar de manera única cada registro por género de película.

```
CREATE_TABLE_MOVIE_BY_GENRE = ""
CREATE TABLE IF NOT EXISTS movies_by_genre (
    movie_id UUID, title TEXT,
    release_year INT, genre TEXT,
    rating FLOAT, director TEXT,
    PRIMARY KEY (genre, rating, release_year)
) WITH CLUSTERING ORDER BY (rating DESC, release_year DESC);
""
```

Insertar datos de películas en ambas tablas

La misma película debe existir en ambas tablas porque cada tabla está diseñada para responder a un patrón de consulta diferente. Cuando se agregue una nueva película, esta deberá almacenarse en las dos tablas:



Codificación:

Se implementaron por medio del comando INSERT INTO se diseñaron CQL Statements para películas por título y género respectivamente

```
INSERT_MOVIE_TITLE = """
INSERT INTO movies_by_title
(title, release_year, director, genre, rating, movie_id)
VALUES (?, ?, ?, ?, ?, ?);
"""

INSERT_MOVIE_GENRE = """
INSERT INTO movies_by_genre
(movie_id, title, release_year,
genre, rating, director)
VALUES (?, ?, ?, ?, ?, ?);
"""
```

Utilizando Python y el Cassandra Python Driver, se creó una función que permite insertar la información de una película en ambas tablas. Haciendo uso de ambos statements para replicar la información en ambas tablas, también se hace uso de la librería uuid para generar códigos identificadores únicos e irrepetibles para cada película.

```
def insert_movie(session, title, year, director, genre, rating):
    id = uuid.uuid4()
    title_stmt = session.prepare(INSERT_MOVIE_TITLE)
    session.execute(title_stmt, (title, year, director, genre, rating, id))
    genre_stmt = session.prepare(INSERT_MOVIE_GENRE)
    session.execute(genre_stmt, (id, title, year, genre, rating, director))
```

Inserta varias películas y asegúrate de incluir suficientes datos para demostrar que tus consultas funcionan correctamente.

Películas ingresadas a la base de datos:

Título	Año	Director	Género	Rating
Psycho	1960	A. Hitchcock	Horror	8.5
Psycho	1998	G. Van Sant	Horror	4.7
Alien	1979	R. Scott	Sci-Fi	8.5
Coco	2017	L. Unkrich	Animacion	8.4
Drive	2011	N. W. Refn	Accion	7.8
Cats	2019	T. Hooper	Musical	2.8
Minions 2	2025	Gru	Animacion	10

Consultar los datos utilizando diferentes patrones de acceso

Al consultar películas por título, además se solicita el año para poder diferenciar entre dos posibles películas con el mismo título. De esta forma, se puede identificar de forma precisa un registro de la tabla, pues la primary key se compone por las columnas de título y año de lanzamiento.

```
Seleccione opción: 2
Título: Psycho
Año: 1998

Título: Psycho
Año de estreno: 1998
Director: G. Van Sant
Género: Horror
Rating: 4.7
```

```
Seleccione opción: 2
Título: Psycho
Año: 1960

Título: Psycho
Año de estreno: 1960
Director: A. Hitchcock
Género: Horror
Rating: 8.5
```

Actualizar información de una película

Para actualizar la información de una película en ambas tablas, se deben construir dos queries de tipo UPDATE adaptadas a la estructura de cada una de ellas.

```
UPDATE_MOVIE_DIRECTOR_TITLE = """"
UPDATE movies_by_title
SET director = ?
WHERE title = ? AND release_year = ?;
""""

UPDATE_MOVIE_DIRECTOR_GENRE = """"
UPDATE movies_by_genre
SET director = ?
WHERE genre = ? AND rating = ? AND release_year = ?;
""""
```

Estos queries son usados en una función que recibe como parámetros valores para todas las columnas que componen las primary keys de cada una de las tablas. Una vez que los statements son preparados con los argumentos correspondientes, son añadidos a un batch para cuidar la integridad de la información, de manera que las actualizaciones solo se hagan efectivas cuando los UPDATES de ambas tablas se hayan completado correctamente.

```
def update_movie_director(session, title, genre, release_year, rating, new_director):
    title_stmt = session.prepare(UPDATE_MOVIE_DIRECTOR_TITLE)
    genre_stmt = session.prepare(UPDATE_MOVIE_DIRECTOR_GENRE)
    # Usar batches para asegurar que ambas tablas están actualizadas
    batch = BatchStatement()
    batch.add(title_stmt, (new_director, title, release_year))
    batch.add(genre_stmt, (new_director, genre, rating, release_year))
    try:
        session.execute(batch)
        print("Película actualizada")
    except Exception as e:
        print(f"Error al actualizar la película: {e}")
```

¿Fue posible realizar la actualización?, ¿por qué?

La actualización se pudo realizar después de hacer algunos cambios a la estructura de la tabla y aumentando la cantidad de información requerida para hacer la actualización. Para los UPDATES de Cassandra, se debe especificar un valor para cada una de las columnas que conforman las primary keys de las tablas. Entonces, al

agregar una columna a la clustering key de la tabla que organiza las películas por género, esta tuvo que ser agregada también en la condición del UPDATE.

- ¿director forma parte de la llave primaria?

El campo de director no forma parte de la primary key, por esto puede ser actualizado. Las actualizaciones de valores en la primary key no están permitidas en Cassandra, pues haría inservible la organización en particiones y el ordenamiento en ellas si en cada actualización los registros pudieran cambiar de partición o de posición en ellas, dificultando en gran medida su localización.

- ¿Qué información necesita Cassandra para identificar el registro que se desea actualizar?

Cassandra necesita valores para todas las columnas que componen la primary key de una tabla, pues la partition key ayuda a ubicar el registro a actualizar en un nodo y en una partición específica, y la clustering key hace posible encontrarlo dentro de la partición a la que pertenece.

En la siguiente captura se observa el proceso de actualización de una película:

```
Seleccione opción: 4
Título: Minions 2
Año: 2025
Género: Animation
Rating: 10
Nuevo Director: Papoi
Película actualizada
```

En consecuencia, se puede observar que el nuevo director ahora aparece en ambas tablas.

```
Seleccione opción: 2
Título: Minions 2
Año: 2025

Título: Minions 2
Año de estreno: 2025
Director: Papoi
Género: Animation
Rating: 10.0
```

```
Seleccione opción: 3
Género: Animation
Minions 2 (Papoi) - 10.0
Coco (L. Unkrich) - 8.4
```

- ¿Cassandra puede actualizar automáticamente el registro correspondiente en ambas tablas?

No, pues según la documentación de Cassandra, las actualizaciones son “aplicadas en cada partición de forma atómica y aislada” (<https://cassandra.apache.org/doc/latest/cassandra/developing/cql/dml.html#update-statement>), es decir, que no se realizan actualizaciones parciales, sino que todos los nuevos valores en un UPDATE son actualizados si este es exitoso, y que únicamente tiene efecto en la partición en la que se encuentre el o los registros a actualizar. Dado que los registros, al estar en distintas tablas pertenecen a distintas particiones, es necesario hacer dos UPDATEs para actualizar los datos en ambas particiones.

- ¿Qué ocurre cuando la misma información se almacena en múltiples tablas?

Cuando la información se almacena en distintas tablas, se pueden hacer consultas de forma más rápida, pues cada una de las tablas puede optimizarse para un tipo de consulta, pero los datos deben ser insertados, eliminados o actualizados en todas las tablas para mantener íntegra la información.

- ¿Cómo debe manejar la aplicación las actualizaciones de información desnormalizada?

En la aplicación se deben usar varios statements adaptados a la estructura de cada tabla para insertar, eliminar, actualizar y consultar la información en cada una de ellas.

- ¿Por qué es importante este comportamiento al diseñar aplicaciones con Cassandra?

Este comportamiento es importante porque permite aprovechar los mecanismos que utiliza Cassandra para almacenar y consultar los datos. Aunque almacenar datos desnormalizados hace más costosa la escritura de los datos al tener que tomar en cuenta múltiples primary keys y escribiendo en distintas tablas al agregar una nueva instancia de alguna entidad, la consulta se facilita, pues cada tabla optimiza una consulta específica, organizando en cada una la información en particiones distintas en las que los datos están ordenados exactamente como lo requiere la consulta.

Eliminar los datos y cerrar la conexión

Para realizar una correcta eliminación de datos es necesario usar el comando DELETE FROM especificando el nombre de la tabla y seleccionando toda la PK para que así el programa pueda hacer uso posterior de la función delete_movie

```
DELETE_MOVIE_TITLE = """"
DELETE FROM movies_by_title
WHERE title = ? AND release_year = ?;
""""

DELETE_MOVIE_GENRE = """"
DELETE FROM movies_by_genre
WHERE genre = ? AND rating = ? AND release_year = ?;
""""
```

La función delete_movie hace un llamado a ambos DELETE tanto para título como para género, es necesario hacer uso de un lote de operaciones batch para permitir la eliminación de toda la información de ambas tablas, es necesario ya que Cassandra hace uso de la desnormalización ya que se adapta a guardar los mismos datos organizados por diferentes claves de partición para permitir distintas consultas, es por eso que batch agrupa múltiples operaciones para poder enviarlas en una sola petición a la base de datos. A su vez con session.execute(batch) hace una transacción atómica donde nos aseguramos que ambas tablas se borren o de lo contrario ninguna lo haga.

```
def delete_movie(session, title, genre, rating, release_year):
    title_stmt = session.prepare(DELETE_MOVIE_TITLE)
    genre_stmt = session.prepare(DELETE_MOVIE_GENRE)
    batch = BatchStatement()
    batch.add(title_stmt, (title, release_year))
    batch.add(genre_stmt, (genre, rating, release_year))
    try:
        session.execute(batch)
        print("Película eliminada")
    except Exception as e:
        print(f"Error al eliminar la película: {e}")
```

Ejemplo de eliminación de película desde título como género:

```
=== Movie Database Menu ===
1. Insertar película
2. Consultar por título
3. Consultar por género
4. Actualizar director
5. Eliminar película
0. Salir
Seleccione opción: 5
Título: Minions 2
Año: 2025
Género: Animacion
Rating: 10
Película eliminada
```

Después de eliminar todas las películas de la base de datos podemos salir de nuestro ambiente virtual y detener el docker de Cassandra, para comprobar se puede introducir el comando docker ps que lista todos los contenedores activos

```
sofiajimenez@192 movies-sofia-haziel % source "/Users/sofiajimenez/Documents/5to semestre/BDNR/n
sofia-haziel/.venv/bin/activate"
(.venv) sofiajimenez@192 movies-sofia-haziel % deactivate
sofiajimenez@192 movies-sofia-haziel % docker stop cassandra-db
cassandra-db
```

```
sofiajimenez@192 movies-sofia-haziel % docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
sofiajimenez@192 movies-sofia-haziel %
```